GraphModi: Updating Graph Language Models in Multi-turn Dialogue
Final Report

Prasoon Dev Yajat Lakhanpal Hrishiraj Mitra Arihant Rastogi
Team Hansraj
International Institute of Information Technology, Hyderabad

Code: https://github.com/Yajat31/Graph-Modi
Checkpoints: https://huggingface.co/buckets/prasoon50/graphmodi-v4_1

Abstract

Graph language models (GLMs) answer questions from soft graph tokens, where the graph is assumed to be fixed. In dialogue, users revise the world, and thus, the graph tokens computed before become stale. We build a synthetic benchmark of multi-turn sessions, where each turn is an update followed by a question and many updates change the answer. GraphModi (our proposed solution) converts each revision into a typed edit program, applies it to an explicit graph, re-encodes the result, and answers from the new graph tokens with a frozen Llama-3.1-8B-Instruct, never seeing the revision text. Accuracy from the re-encoded graph stays the same across session lengths, while accuracy from stale graph tokens falls as soon as the graph is revised, most sharply on turns where the edit changes the answer. Giving the model the revision text does not repair stale tokens, and graph-blind and text-only baselines stay well below re-encoding. Edits written by the frozen model itself cost almost nothing, but the readers track local facts about the queried stations rather than reasoning over longer paths. These results motivate the need of such a framework which allows a GLM to re-encode its own input for maintaining correct chat history and updates.

1 Introduction

Graph language models (GLMs) map a graph through a GNN into soft tokens that a frozen or adapted language model can attend to (Perozzi et al., 2024; Wang et al., 2024; He et al., 2024; Plenz and Frank, 2024). All such work assumes the graph does not change. Real graphs are updated, and in dialogue a user may report changes as they happen. After such a change, graph tokens computed from the old graph describe a world that no longer exists.

One way to fix this is to keep the updates in the text context and rely on the language model to track them. Dynamic-graph QA benchmarks study this setting with graphs described in text (Comrie and Kleinberg, 2025). Another is to update stored knowledge directly, but parametric knowledge editors can recall a new fact and fail to propagate its effects (Zhong et al., 2023). GraphModi keeps state external: an edit program is applied to an explicit graph, the graph is re-encoded, and the QA call receives only the new graph tokens and the question, never the revision text. This lets us score writing (did the edit land?) separately from reading (can the model answer from the new tokens?).

Research questions.

RQ1: Can a frozen LLM emit executable edit programs from a few-shot prompt, and what do its errors cost downstream?

RQ2: Does answering from re-encoded tokens beat stale G_0 tokens and text-history controls as turns accumulate, especially on answer-changing turns?

RQ3: Do graph tokens carry information beyond graph-blind baselines, and what do the GNNs actually pass to the LLM?

Contributions.

i) A multi-turn benchmark for updating GLMs: synthetic metro graphs with sessions of revisions and questions across graph size, density, session length, and task, gold edit programs and answers checked by independent solvers, and enough answer-changing turns that ignoring the edits does not pay.

ii) Paired evaluation conditions on the same sessions (stale tokens, revision text, a shuffled graph, a graph-blind soft prompt, zero-shot serialized graphs, and local-shortcut ceilings) that score writing separately from reading and show when a model actually uses the current graph.

iii) The main trend: accuracy from re-encoded tokens stays steady over a session while stale tokens fall, most on answer-changing turns, and adding the revision text does not close the gap.

iv) Further trends: model-written edits cost almost nothing, every reader reads local facts rather than longer paths, and how the GNN is obtained matters more than training it, since a frozen random GNN reads as well as a trained one while contrastive pretraining hurts.

2 Related Work

Graphs as text. A graph can be written in natural language so that an LLM attempts structural reasoning with no GNN (Wang et al., 2023; Fatemi et al., 2024). Fatemi et al. (2024) show that accuracy varies substantially with how the graph is encoded as text, and Wang et al. (2023) find that LLM graph reasoning can rely on spurious patterns instead of the actual relationships in the graph. We evaluate the serialized graph as a zero-shot baseline, and several of our controls use text history.

Soft graph tokens. GraphToken learns a graph encoder end-to-end through the language-modelling objective with a frozen LLM (Perozzi et al., 2024). TEA-GLM pretrains a GNN with graph contrastive learning (Zhu et al., 2020) plus a loss that aligns it with the LLM's token embeddings, freezes it, and then trains a linear projector onto a fixed number of tokens (Wang et al., 2024). G-Retriever and the graph language models of Plenz and Frank (2024) also couple graph representations with frozen or lightly adapted LMs (He et al., 2024). All of them evaluate QA against a graph supplied once per query and do not explore how a revision should modify the encoded graph. We build TEA-style and GraphToken-style readers rather than claim exact reproductions, since our readouts, tasks, and training data differ from the originals.

Do GLMs use the graph? CLEGR shows that a graph-blind soft-prompted LLM can rival full GLMs on metro reasoning (Petkar et al., 2026), and Liu et al. (2024) ask the same question for soft prompting on graph learning tasks. In CLEGR every model, including the soft prompt, also reads the graph as text; our soft prompt does not, and we test reading the graph as text separately. We adopt the soft-prompt baseline, add a shuffled-graph control, and treat parity with either as evidence against a graph-reading claim.

What a GNN can count. Xu et al. (2019) show that sum aggregation can tell apart neighbourhoods that mean and max aggregation cannot, for example neighbourhoods of different sizes. Kipf and Welling (2017) observe that even an untrained GCN with random weights gives useful node embeddings. Both facts turn out to explain our readers (Section 7).

Dynamic graphs and stateful dialogue. DyGraphQA evaluates LLMs on graphs that change through additions and deletions described as text or structured data (Comrie and Kleinberg, 2025). MQuAKE shows that knowledge edits made inside model parameters often fail to propagate to multi-hop questions (Zhong et al., 2023). SKG-Eval stores conversational claims in an incremental knowledge graph to detect cross-turn conflicts (Shil and Samui, 2026).

3 Problem Formulation

Consider a transit assistant that initially knows two routes between a pair of stations. The user then says that one station is closed. The text decoder sees the correction, but a frozen graph representation still describes the station as open, so the model is given two versions of the world. Keeping the full conversation in the prompt may help, but it does not repair the graph that the GNN encodes, and it makes the system depend on implicit state tracking by the language model. Updating needs *writing* (mapping a revision to a graph change) and *reading* (answering from re-computed graph tokens), which we score separately on turns where the edit changes the answer and where it does not.

Graph and session. Let $G = (V, E, a, \rho)$ be an undirected attributed metro graph. Node attributes include line, accessibility, architecture, cleanliness, music, size, and $\mathrm{status} \in \{\mathrm{open}, \mathrm{closed}\}$; edge relations ρ are track or $\mathrm{transfer}$. A session is $S = (G_0, (r_t, q_t)_{t=1}^{T})$, where r_t is a natural-language revision and q_t a question.

Edit language. An edit program is a semicolon-separated list of at most three operations terminated by `END`. Legal operators are `SET NODE` (set station status), `ADD EDGE`, `DEL EDGE`, and `NOOP`. Gold programs $P_t^\star$ induce

$$G_t = \mathrm{Apply}(G_{t-1}, P_t^\star). \tag{1}$$

Gold and stale answers are $a_t^\star = \mathrm{Solve}(G_t, q_t)$ and $\tilde{a}_t = \mathrm{Solve}(G_{t-1}, q_t)$; the answer-changing set is $\mathcal{C} = \{t : a_t^\star \neq \tilde{a}_t\}$.

GraphModi. At each turn the system computes

$$\hat{P}_t = f_{\text{edit}}(r_t), \tag{2}$$

$$\hat{G}_t = \mathrm{Apply}(\hat{G}_{t-1}, \hat{P}_t), \tag{3}$$

$$Z_t = W \cdot \rho\big(\mathrm{GNN}(X(\hat{G}_t)), q_t\big) \in \mathbb{R}^{K \times d}, \tag{4}$$

$$\hat{a}_t = \mathrm{LLM}\big(\mathrm{prompt}(q_t),\, Z_t\big). \tag{5}$$

Edit prediction uses the revision text only; the graph is used for node-name resolution. The revision r_t is withheld from the QA call, so a correct answer after an edit has to come from the updated tokens. The LLM stays frozen; the GNN and projector are trained by minimizing $-\log p(a \mid q, Z(G, q))$ on static QA pairs.

Metrics. We report exact-match accuracy overall, on $\mathcal{C}$, and on its complement, plus execution-equivalent edit accuracy for the write path. Confidence intervals are 95% session-clustered bootstrap intervals, and differences between conditions are paired by turn. Two reference points run no model and are not competitors. The *per-task majority* always gives each task's most common gold answer. The *stale oracle* answers every question perfectly, but on the graph before the turn's edit: it returns $\tilde{a}_t$. It is therefore right on every turn whose answer the edit leaves unchanged and wrong on every answer-changing turn, so its overall score is simply the share of unchanged turns. It shows how much a benchmark rewards ignoring edits, and it scores above every reader because it never makes a reading error (Appendix B).

4 Methodology

Write path. A few-shot prompt (output format, a NOOP example, multi-operation outputs separated by '`;`' and ended by '`END`') makes the frozen LLM emit an edit program with greedy decoding. Station names are mapped to node IDs and operations are applied as given; invalid operations are skipped, so an edit error carries over to later turns. Edit accuracy is the fraction of turns where the predicted program yields the same graph as the gold program.

Read path. Nodes are featurized by a frozen `bert-base-uncased` encoder (Devlin et al., 2019) over the node text "`label: attr=val, ...`" (768-d, status included) and passed through a 3-layer GraphSAGE (Hamilton et al., 2017). Each layer computes

$$h'_v = \mathrm{LN}\Big(W_s h_v + W_n \textstyle\sum_{u \in N(v)} h_u\Big), \tag{6}$$

with a ReLU between layers and widths 768→1024→1024→4096. The neighbours are *summed*, not averaged; Section 7 explains why this matters. The readout takes the source and target node embeddings when the question names them and the mean pool otherwise, adds a hop-radius scalar and a task one-hot, and a linear map projects it to $K{=}10$ tokens of width 4096. The text of the queried station(s) is added to the question with status removed, so status can only come from the graph. The LLM is a frozen Llama-3.1-8B-Instruct in bfloat16 (Llama Team, AI @ Meta, 2024).

Readers. All graph readers share this encoder shape and differ in how the GNN is obtained. *GraphToken-style* starts the GNN from random weights and trains it jointly with the projector. *TEA-style* first pretrains the GNN with TEA-GLM's self-supervised objective, following the authors' code: the GNN sees two copies of each graph with some edges dropped and some features masked, and learns to give each node the same embedding in both copies, plus a loss that aligns node embeddings with the principal directions of Llama's token embeddings. The GNN is then frozen and only the projector is trained. Two ablations separate these choices: a *warm start* trains the TEA-GLM-pretrained GNN jointly, and a *frozen random GNN* trains only the projector on top of a random GNN. The *soft prompt* learns 10×4096 prompt vectors with no GNN.

Training. Every reader trains for one epoch on 24,000 static QA pairs, including before/after pairs around a random edit, and the checkpoint with the lowest validation loss is evaluated. The projector learning rate was chosen per reader on validation loss. No model sees multi-turn sessions in training; all must generalize from static examples to graph states created at evaluation time. Appendix F lists all hyperparameters.

Benchmark. Graphs are small synthetic metro networks of 12–22 stations: a ring of stations with extra chords and a few side chains, some of which end in a small loop (Appendix G). Three density

Graph size	12–22 stations
Mean degree (sparse/med./dense)	2.8 / 3.8 / 5.0
Max diameter	7 / 6 / 5
Static eval (val / test)	2,178 / 4,356
Sessions / turns per split	1,188 / 4,455
Multi-turn tasks	9
Answer-changing turns (test)	36.4%
NOOP turns (test)	16.7%
Training pairs	24,000, 1 epoch
Seeds	3 (GT, TEA); 1 (others)

Table 1: Benchmark and training setup.

settings control the mean degree and cap the diameter, so every station is within a few GNN hops of any other. Nodes carry CLEGR-like attributes and fictional two-word station names. Static QA covers eleven task families (Appendix E) with per-task answer quotas, so yes/no tasks are balanced. The multi-turn grid crosses 11 node counts, 4 session lengths $\{1, 2, 4, 8\}$, 3 densities, and 9 tasks. A quota makes 36% of test turns answer-changing, and every stored label and edit was re-derived by independent solvers.

Local shortcuts. Because the benchmark is synthetic, we measure how far simple features of the queried stations go. A per-task classifier that sees only one- and two-hop features (degree, status, closed and leaf neighbours, counts at two hops, and similar) reaches 67–70% on the structural tasks (Appendix H). A 3-layer GNN sees all of these features, so accuracy near these ceilings shows local reading, not multi-hop reasoning. The 1,052 test turns that these classifiers get wrong form the *local-wrong* slice.

Conditions. Each multi-turn session is evaluated under the paired conditions in Table 2. They differ in which graph the tokens encode and in what text the model sees. The gap between the oracle and predicted conditions is the cost of edit errors.

5 Experiments

Table 1 summarizes the setup. All readers share the frozen Llama-3.1-8B decoder, the BERT node features, the encoder shape, and the same session IDs. Multi-turn results are on the 4,455 test turns; the validation split is used only for checkpoint and learning-rate selection. GraphToken-style and TEA-style are averaged over three seeds; the ablations and baselines use one seed.

6 Results

6.1 Static evaluation

Table 3 shows single-graph QA (RQ3). Every graph reader beats the soft prompt, and the ranking is the same on validation. GraphToken-style and the frozen random GNN are level. Their gains come from structural facts such as node degree and cycle membership, which are also the tasks a local classifier solves well. The two readers built on the TEA-GLM-pretrained GNN fall behind, but they change *which* tasks are solved rather than losing accuracy everywhere: they are much weaker on node degree and cycle membership and stronger on attribute questions (Appendix E).

6.2 Multi-turn conditions

Table 4 gives the multi-turn results. For every reader, re-encoding the gold graph is the best condition, and re-encoding the graph built from the model's own edits is within 0.1 points of it. For GraphToken-style, re-encoding is about 13 points above stale tokens, with or without the revision text, and 22 points above a shuffled graph, on every seed (paired intervals in Appendix I). The graph-blind channels stay far below: the soft prompt that reads every revision reaches 34.7%, and zero-shot Llama reading the current graph as text does no better than the per-task majority (Appendix L). The frozen random GNN is level with GraphToken-style. TEA-style and the warm start reach only 42–45%; they still gain from re-encoding, but less. The stale oracle (63.6%) is above every reader only because it reads the old graph perfectly; on answer-changing turns it scores 0% (Section 6.4).

6.3 Multi-turn degradation

Figure 1 addresses RQ2 (numbers in Appendix I). Re-encoded tokens hold their accuracy at every session length. Stale tokens lose about six points as soon as a session has more than one turn, and adding the revision text does not bring them back. They do not keep falling in longer sessions, because the benchmark keeps the share of answer-changing turns about the same at every length, so a later edit is as likely to restore an answer as to change it. The soft prompt rises slightly with session length, since the accumulated revision text answers more questions directly. TEA-style shows the same ordering of conditions at a lower level.

Condition	Explanation
`oracle_updated_graph`	Graph tokens encode the gold G_t, re-encoded every turn; the prompt is the question only. The tokens describe the true current graph, with no edit-prediction error.
`predicted_updated_graph`	The frozen LLM writes an edit program from the revision text, the executor applies it to its own previous graph, and the result is re-encoded every turn; the prompt is the question only. Edit errors persist into later turns. This is the full GraphModi pipeline.
`structure_only`	Graph tokens encode the initial graph G_0 and are never updated; the prompt is the question only.
`frozen_graph_history`	Stale G_0 tokens with the accumulated revision text added to the question. Tests whether text can make up for stale tokens.
`shuffled_graph`	Tokens encode another session's G_0: right token format, wrong graph. Tests whether the tokens encode this session's graph.
`question_only`	No graph tokens and no history; the prompt is the question only.
`soft_prompt`	Trained soft prompt, no GNN; the prompt is the revision history plus the question. A graph-blind baseline that can answer many questions from the revision text.
`soft_prompt_no_history`	The same soft prompt with the question only.
`serialized_current_graph`	Zero-shot Llama, no graph tokens; the prompt is the gold G_t written as text plus the question.
`serialized_initial_history`	Zero-shot Llama; G_0 as text, the revision history, and the question.
`token_matched_history`	Zero-shot Llama; the last 64 words of revision history plus the question.
stale oracle	No model is run. A perfect answer computed on the graph before the turn's edit ($\tilde{a}_t$): right on every unchanged turn and wrong on every answer-changing turn. Shows what ignoring edits is worth on this benchmark.
per-task majority	No model is run. Returns each task's most common gold answer.

Table 2: Multi-turn evaluation conditions. "History" is the accumulated revision text of earlier turns. The last two rows are reference points, not model conditions.

	GT	TEA	Warm	Rand.	Soft	Maj.
Validation	51.1	43.2	47.1	51.5	33.7	–
Test	50.3	42.7	46.8	50.8	32.6	29.6

Table 3: Static exact-match accuracy (%). GT: GraphToken-style; TEA: TEA-style; Warm: warm start; Rand.: frozen random GNN; Maj.: per-task majority. GT and TEA are means over three seeds.

Condition	GT	TEA	Warm	Rand.
oracle_updated_graph	54.3	42.1	44.5	55.2
predicted_updated_graph	54.2	42.1	44.6	55.1
frozen_graph_history	41.2	38.2	38.8	41.7
structure_only	41.2	37.9	38.0	41.7
shuffled_graph	32.5	31.9	30.8	31.8
question_only		26.5		
soft_prompt / no history		34.7 / 33.3		
serialized_current_graph		31.0		
serialized_initial_history		29.4		
token_matched_history		27.9		
References (no model is run)				
per-task majority		31.8		
stale oracle		63.6		

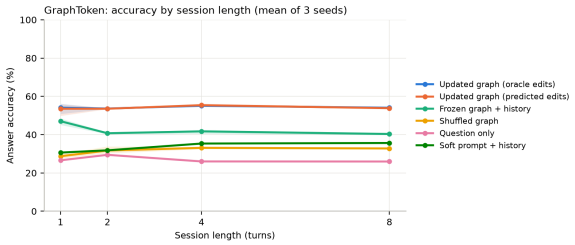
Table 4: Multi-turn test accuracy (%), 4,455 turns. GT and TEA are means over three seeds (seed s.d. of the oracle row: 0.2 and 0.4). Rows below the first rule do not depend on the graph reader. The stale oracle is perfect on unchanged turns and scores 0% on answer-changing ones, so it is a reference, not a competitor (Appendix B).

6.4 Answer-changing turns

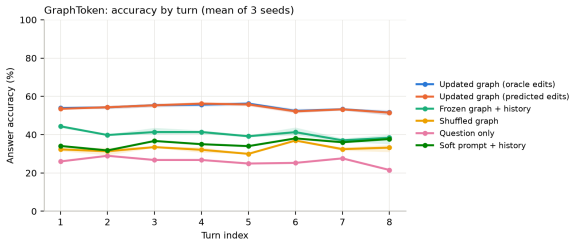
Splitting turns by whether the edit changed the answer gives the clearest picture (Table 5; Figure 2 in Appendix A). Where the answer did not change, stale and re-encoded tokens are close. Where it did, stale tokens fall below the per-task majority, while re-encoded tokens keep almost all of their accuracy, a gap of about 28 points. The gap remains on changed turns whose question does not name an edited station. On these turns the revision text cannot simply be matched to the question: a text lookup that reaches 79% overall scores 2.5% here (Appendix H). The frozen random GNN tracks edits as well as GraphToken-style. TEA-style gains much less, and on the unmentioned turns its re-encoded tokens are no better than its stale ones.

6.5 Density, task, and local shortcuts

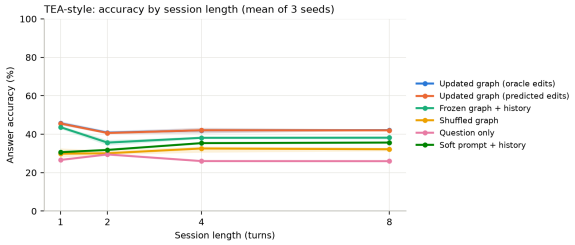
Accuracy changes little with density, and re-encoding stays more than 20 points ahead of the shuffled graph at every density (Appendix I). By task, re-encoding helps most where the answer is a local structural fact that an edit changes: node



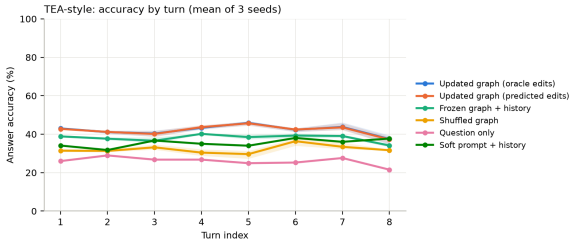
(a) GraphToken-style, by session length



(b) GraphToken-style, by turn index



(c) TEA-style, by session length



(d) TEA-style, by turn index

Figure 1: Multi-turn test accuracy by session length (left) and by turn index (right), for GraphToken-style (top) and TEA-style (bottom), means over three seeds with the seed range shaded. Later turns occur only in 8-turn sessions (297 sessions).

Condition	Changed	Unchanged	Unmentioned
GT oracle	51.8	55.7	38.3
GT predicted	51.2	55.9	38.2
GT stale + history	24.2	50.9	26.8
GT shuffled	29.4	34.2	27.8
Random GNN oracle	52.8	56.6	40.9
TEA oracle	37.8	44.6	26.3
TEA stale + history	29.1	43.4	27.5
soft_prompt	32.2	36.1	24.8
serialized_current_graph	27.3	33.2	27.6
per-task majority	27.3	34.4	25.3
stale oracle	0.0	100.0	0.0

Table 5: Test accuracy (%) on answer-changing turns (1,622), unchanged turns (2,833), and answer-changing turns whose question names no edited station (359). The last two rows are references that run no model. Every reader and condition is in Appendix A.

degree, cycle membership, and edge existence. On path questions (reachability, shortest path) every reader stays far below what a local classifier reaches, and on the 1,052 local-wrong turns every graph reader is at or below the question alone. The readers therefore use the graph to track local facts such as degree, status, and nearby closures, which is what most edits change, but they show no reasoning beyond a two-hop neighbourhood.

6.6 Write path

The `predicted_updated_graph` condition answers RQ1. The frozen LLM writes a correct edit program on 96.6% of test turns. Because an error persists for the rest of a session, the graph built from its own edits is exactly right on 91.1% of turns. The weakest case is a lone `ADD EDGE`, and the `notice` wording, held out of the few-shot prompt, is harder than the two wordings shown in it (Appendix J). Even so, edit errors cost at most 0.1 points for every reader.

6.7 What the readers learn

To see what each GNN hands to the projector, we fit linear probes on its frozen node embeddings for three local facts the questions depend on: a station's degree, its status, and the number of stations within two hops (method and full table in Appendix K). Three trends stand out. First, the BERT input features contain status, which is written in the node text, but not degree. Second, a frozen random GNN with sum aggregation makes all three facts linearly readable, and joint training by GraphToken-style adds almost nothing. Third, the TEA-GLM-pretrained GNN drops status to the majority-class rate and weakens degree and the two-hop count; the warm start recovers degree but not status. The loss is not embedding collapse: the TEA-GLM embeddings are the most spread out, while the random and GraphToken-style embeddings are nearly collapsed and still carry every fact. Section 7 explains these results.

7 Inferences

Graph tokens encode the specific, current graph. Re-encoded tokens beat a shuffled graph, a graph-blind soft prompt with or without the revision text, and zero-shot Llama reading the serialized graph, at every density and session length and on all three seeds. A model that ignored the tokens would score the same under all of these conditions, so the tokens carry session-specific information (RQ3).

Staleness hurts exactly where an edit matters. On answer-changing turns, stale GraphToken-style tokens fall below the per-task majority while re-encoded tokens keep their accuracy; on unchanged turns the two are close. The gap survives on changed turns whose question does not name the edited station, where no text lookup helps (RQ2).

The gain comes from re-encoding, not from text history. Adding the revision text to stale tokens does not help: `frozen_graph_history` and `structure_only` score the same. Text channels that see the full history do not catch up either. Updating the graph and reading it again is what recovers the answer.

The write path is not the bottleneck. Model-written edits are almost always correct and cost almost nothing (RQ1), but revisions are templated, so this likely overstates the write path on free-form dialogue.

The reader is local. Accuracy stays at or below learned local ceilings on the structural tasks, and on turns that no local rule solves it is no better than the question alone. The benchmark rewards tracking local facts that updates change; it does not show that soft graph tokens support multi-hop reasoning.

Why a random GNN is enough: sum aggregation counts. Each GNN layer adds a station's own vector to the sum of its neighbours' vectors and then normalizes the result. Two facts make this informative even when the weights are random (Kipf and Welling, 2017). First, the BERT vectors of different stations are very similar, because their node texts share most words (mean cosine 0.97 within a graph). The neighbour sum is therefore close to *the number of neighbours times a shared vector*. The more neighbours a station has, the more this shared part outweighs the station's own vector, so the direction of the output changes with degree. Layer normalization rescales the vector but keeps this direction, and the second and third layers repeat the effect for stations two and three hops away. Second, a random linear map into a wider space keeps different inputs apart, so a difference that is written in the node text, such as open versus closed, survives the layers. A linear projector can then read degree, status, and nearby counts without any GNN training, which is what the probes show. This also explains why the random and GraphToken-style embeddings look nearly collapsed: since stations look alike, most of the variation left between them is the count itself, which is exactly what the tasks need. Training the GNN jointly cannot add much, because the information is already linearly available; the limit is how much the frozen LLM can read from ten soft tokens. Mean aggregation would remove the count, because dividing the sum by the degree makes a station with two similar neighbours look like one with five (Xu et al., 2019). Our midterm readers used mean aggregation and stayed below 20% on node degree; with sum aggregation GraphToken-style answers almost every degree question, although the two runs also differ in benchmark version and training.

Why contrastive training weakens the degree signal. TEA-GLM pretraining shows the GNN two copies of each graph, one with 30% and one with 40% of the edges dropped, and trains it to give each station the same embedding in both copies while keeping different stations apart. Dropping edges changes a station's degree, and by a different amount in each copy. Any part of the embedding that follows the exact neighbour count therefore differs between the two copies, and the loss pushes it down. The GNN learns to describe a station by what survives edge dropping, mainly its own text, rather than by how many neighbours it has. We think status is lost for a related reason: only one to three stations per graph are closed, so status does little to tell stations apart, and the objective has no reason to keep it. The degree signal is weakened rather than erased. A probe that rescales every dimension still finds it, but the projector reads the raw embedding, where the degree direction most likely competes with the many directions that carry node text (the TEA-GLM embeddings spread their variance over about eight

times more directions). TEA-style accordingly answers only about half of the multi-turn degree questions, against almost all for the frozen random GNN. Starting joint training from this GNN (the warm start) brings degree back in the probe, but only partly on degree questions, and does not recover status, so one epoch at a small learning rate undoes only part of the pretraining. In short, the pretraining objective teaches the encoder to ignore the edge-level details that updates change.

What TEA-GLM pretraining keeps. The second TEA-GLM loss pulls node embeddings towards the main directions of Llama's token embeddings. This seems to keep node-text content, such as line and architecture, in a form the projector can map to word-like tokens: both readers built on this GNN are the strongest on attribute questions. We did not probe attributes directly, so this part is a hypothesis. For GLMs over graphs that change, the encoder must keep exactly the local details that augmentation-based contrastive objectives are designed to ignore.

8 Conclusion

Graph tokens go stale once a dialogue revises the graph they were computed from. We built a multi-turn benchmark for this setting, with paired conditions that score writing an edit separately from reading the new graph. On it, a frozen LLM writes edit programs almost perfectly, and answering from the re-encoded graph keeps accuracy steady across a session, while stale tokens lose accuracy exactly on the turns where an edit changes the answer. Giving the model the revision text does not repair stale tokens, so keeping the graph outside the model and re-encoding it is what keeps answers current. The readers, however, track local facts about the queried stations rather than reasoning over longer paths. How the GNN is obtained matters more than training it: a sum-aggregation GNN with random weights already exposes the counts the tasks need, while contrastive pretraining that drops edges teaches the encoder to ignore them. Encoders for graphs that change should therefore be trained to keep the edge-level details that updates touch. Natural next steps are free-form rather than templated revisions, questions that need multi-hop reasoning beyond local shortcuts, and a contradiction probe in which a session drifts into a logically inconsistent state (A is the father of B, B of C, and C of A) that the GLM should flag.

Limitations

The benchmark is synthetic and its revisions follow templates, so edit accuracy likely overstates performance on free-form dialogue. GraphToken-style and TEA-style use three seeds; the warm start, the random GNN, and the soft prompt use one, so the tie between the random GNN and GraphToken-style rests on a single random-GNN seed. Every reader trains for one epoch on static QA, and longer training could change how the readers rank. Each multi-turn task uses a single revision wording family, so wording effects cannot be separated from task effects. Our explanations of the probe results are consistent with the data but were not tested by targeted ablations, such as a mean-aggregation reader on the current benchmark or an attribute probe. The zero-shot text baselines use a plain question–answer template and see no examples of the answer formats, so they are a floor for reading the graph as text, not its best case. Learned local classifiers reach 67–70% on the structural tasks, so absolute accuracy on this benchmark mostly measures local reading.

References

Cazamere Comrie and Jon Kleinberg. 2025. Can large language models reason on dynamic graphs? In *KDD 2025 Workshop on Structured Knowledge for Large Language Models*.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. Talk like a graph: Encoding graphs for large language models. In *International Conference on Learning Representations*.

Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*.

Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-retriever: Retrieval-augmented generation for textual graph understand-

ing and question answering. In *Advances in Neural Information Processing Systems*.

Thomas N. Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*.

Zheyuan Liu, Xiaoxin He, Yijun Tian, and Nitesh V. Chawla. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM Web Conference 2024*, pages 481–484.

Llama Team, AI @ Meta. 2024. The Llama 3 herd of models. *Preprint*, arXiv:2407.21783.

Bryan Perozzi, Bahare Fatemi, Dustin Zelle, Anton Tsitsulin, Mehran Kazemi, Rami Al-Rfou, and Jonathan Halcrow. 2024. Let your graph do the talking: Encoding structured data for LLMs. *Preprint*, arXiv:2402.05862.

Soham Petkar, K Hari Aakash, Anirudh Vempati, Akshit Sinha, Ponnurangam Kumaraguru, and Chirag Agarwal. 2026. A graph talks, but who's listening? rethinking evaluations for graph-language models. In *Findings of the Association for Computational Linguistics: ACL 2026*.

Moritz Plenz and Anette Frank. 2024. Graph language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4477–4494, Bangkok, Thailand. Association for Computational Linguistics.

Avijit Shil and Suman Samui. 2026. SKG-eval: Stateful evaluation of multi-turn dialogue via incremental semantic knowledge graphs. *Preprint*, arXiv:2605.16650.

Duo Wang, Yuan Zuo, Fengzhi Li, and Junjie Wu. 2024. LLMs as zero-shot graph learners: Alignment of GNN representations with LLM token embeddings. In *Advances in Neural Information Processing Systems*.

Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2023. Can language models solve graph problems in natural language? In *Advances in Neural Information Processing Systems*.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How powerful are graph neural networks? In *International Conference on Learning Representations*.

Zexuan Zhong, Zhengxuan Wu, Christopher Manning, Christopher Potts, and Danqi Chen. 2023. MQuAKE: Assessing knowledge editing in language models via multi-hop questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15686–15702, Singapore. Association for Computational Linguistics.

Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2020. Deep graph contrastive representation learning. In *ICML Workshop on Graph Representation Learning and Beyond*.

A Accuracy on Answer-Changing Turns

Figure 2 and Table 6 split test turns by whether the edit changed the answer, for every reader and condition. The stale oracle is 0% on changed turns and 100% on unchanged ones by definition (Appendix B).

B The Stale-Oracle Reference

What it is. For each turn, the benchmark's symbolic solver answers the question on G_{t-1}, the graph just before the turn's edit, and this answer is scored against the gold answer on G_t; no model is run. The stale oracle is thus a perfect reader that ignores every update. We leave it and the other references out of the figures, since they are not systems and would distort the comparison between readers.

Why it scores above every reader. Most edits do not change the answer to the question asked. On the 2,833 unchanged test turns (63.6%) the pre-edit answer is still correct, and on the 1,622 answer-changing turns it is wrong by definition, so the overall score equals the share of unchanged turns. At every session length it is 61–64%, mirroring the 36–39% share of answer-changing turns (Table 13). A graph reader must both use the current graph and read the answer out of ten soft tokens; the stale oracle is perfect at the second and skips the first. The readers are limited by the second: even with no edits, the best reader reaches only about 51% on static QA, and on unchanged turns, where tracking the edit does not matter, re-encoded GraphToken-style tokens score 55.7% against the oracle's 100%. The gap to the stale oracle is therefore a reading gap, not an edit-tracking gap.

How to read it. The stale oracle measures how much a benchmark rewards ignoring edits. In an earlier version of our generator only 14.6% of turns changed the answer, so the stale oracle scored about 85% and a system that ignored every update would have looked strong; the quota in generator v4.1 raised the share of answer-changing turns to 36% (Appendix G). The comparison that matters is on answer-changing turns (Table 5): there the stale oracle scores 0%, re-encoded GraphToken-style tokens about 52%, and stale GraphToken-style tokens about 23–24%. Stale tokens are the learned, imperfect version of the stale oracle, and they stay above 0% on changed turns because a reader's errors on the old graph sometimes match the new answer, for example on yes/no questions, where getting the old answer wrong means getting the new one right.

C All Readers Together

Figure 3 puts the four graph readers side by side.

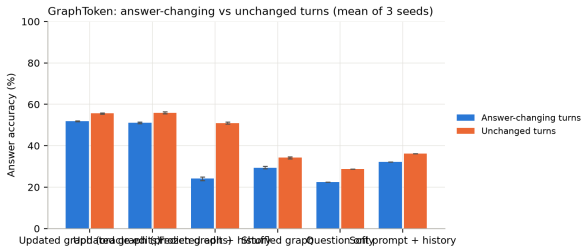
D All Conditions

Figures 4–6 repeat the main-text plots with every condition, for GraphToken-style (top of each figure) and TEA-style (bottom).

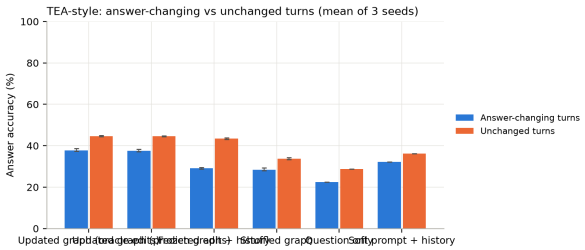
E Task Families

Static QA covers eleven task families; the multi-turn grid uses the first nine. Every answer is computed by a symbolic solver on the current graph, and yes/no tasks are balanced. In the definitions below, A and B are station names.

- **Edge existence**: is there a direct edge between A and B?
- **Reachability**: can B be reached from A using open stations only?
- **Constrained reachability**: can B be reached from A without passing through a station of a given class (no disabled access, no rail, or dirty)? The start and end stations are exempt.
- **Cycle membership**: is A part of any cycle? Asked only about degree-2 stations, so degree alone does not answer it.
- **Node degree**: how many stations are directly connected to A?
- **Filtered neighbour count**: how many direct neighbours of A have a given attribute value?
- **Shortest path**: how many stations, including both endpoints, lie on the shortest open path from A to B (or `unreachable`)?
- **Within-hops count**: how many other stations within two hops of A have a given attribute value?
- **Most common attribute within hops**: what is the most common value of an attribute among the stations within two hops of A, including A itself? Only queries with a unique mode are kept.
- **Attribute lookup**: what is the value of one of A's attributes? The answer is an open categorical value.



(a) GraphToken-style



(b) TEA-style

Figure 2: Test accuracy on turns where the edit changed the answer (36%) and where it did not (64%), means of three seeds; error bars are the seed range.

Reader · condition	Chg.	Unchg.	Unm.	Reader · condition	Chg.	Unchg.	Unm.
GT · oracle	51.8	55.7	38.3	Warm · oracle	40.1	47.0	25.6
GT · predicted	51.2	55.9	38.2	Warm · predicted	40.0	47.2	26.2
GT · stale + history	24.2	50.9	26.8	Warm · stale + history	27.0	45.5	29.5
GT · structure only	23.0	51.7	27.2	Warm · structure only	26.1	44.8	27.3
GT · shuffled	29.4	34.2	27.8	Warm · shuffled	27.6	32.7	25.9
TEA · oracle	37.8	44.6	26.3	Random · oracle	52.8	56.6	40.9
TEA · predicted	37.6	44.6	26.5	Random · predicted	52.1	56.8	40.7
TEA · stale + history	29.1	43.4	27.5	Random · stale + history	24.8	51.4	24.8
TEA · structure only	27.9	43.6	26.6	Random · structure only	22.8	52.5	26.7
TEA · shuffled	28.5	33.8	24.5	Random · shuffled	28.0	34.0	27.3
question_only	22.4	28.8	24.2	serialized_current_graph	27.3	33.2	27.6
soft_prompt	32.2	36.1	24.8	serialized_initial_history	27.2	30.7	28.1
soft_prompt_no_history	29.3	35.6	22.0	token_matched_history	25.5	29.4	24.8
per-task majority	27.3	34.4	25.3	stale oracle	0.0	100.0	0.0

Table 6: Test accuracy (%) on answer-changing turns (Chg., 1,622), unchanged turns (Unchg., 2,833), and answer-changing turns whose question names no edited station (Unm., 359). GT and TEA are means of three seeds (seed s.d. at most 1.5 on Unm. for the oracle rows).

- **Attribute check**: does A have a given attribute value?

The last two are CLEGR's Facts tasks. For them, and for the most-common-attribute task, the queried attribute is removed from the station text in the question, so the answer has to come from the graph tokens. The Facts tasks are static only: revisions set station status or add and remove edges, so they never change a node attribute. Reachability and shortest-path questions use open endpoints only, so status alone does not decide them.

Static accuracy by task family. Table 7 and Figure 7 break Table 3 down by task. GraphToken-style and the frozen random GNN are strongest on node degree, cycle membership, reachability, and edge existence, and stay below the local ceiling on each. On the attribute tasks they stay close to the soft prompt. The two readers built on the TEA-GLM-pretrained GNN have the opposite profile: they are weaker on every structural task (TEA-style is at chance on cycle membership) but are the strongest on attribute lookup and the most common attribute. Attribute check stays near the majority for every reader.

F Hyperparameters and Training

TEA-GLM pretraining. Pretraining follows the authors' released code. Each batch of 32 graphs is seen in two views: edges are dropped with probability 0.3 and 0.4, and node-feature dimensions are masked with probability 0 and 0.1. An instance-wise GRACE loss (Zhu et al., 2020) through a two-layer projection head pulls the two views of each node together and pushes other nodes apart. A feature-wise contrastive loss acts on the node embeddings projected onto the top 1,000 principal components of Llama's token-embedding table. Training ran all 70 epochs (best epoch 68); the training loss fell from 12.80 to 10.11 (instance term 6.46 to 4.76, feature term 6.34 to 5.35), and the held-out loss was 9.90. One pretrained GNN is shared by all TEA-style seeds and the warm start.

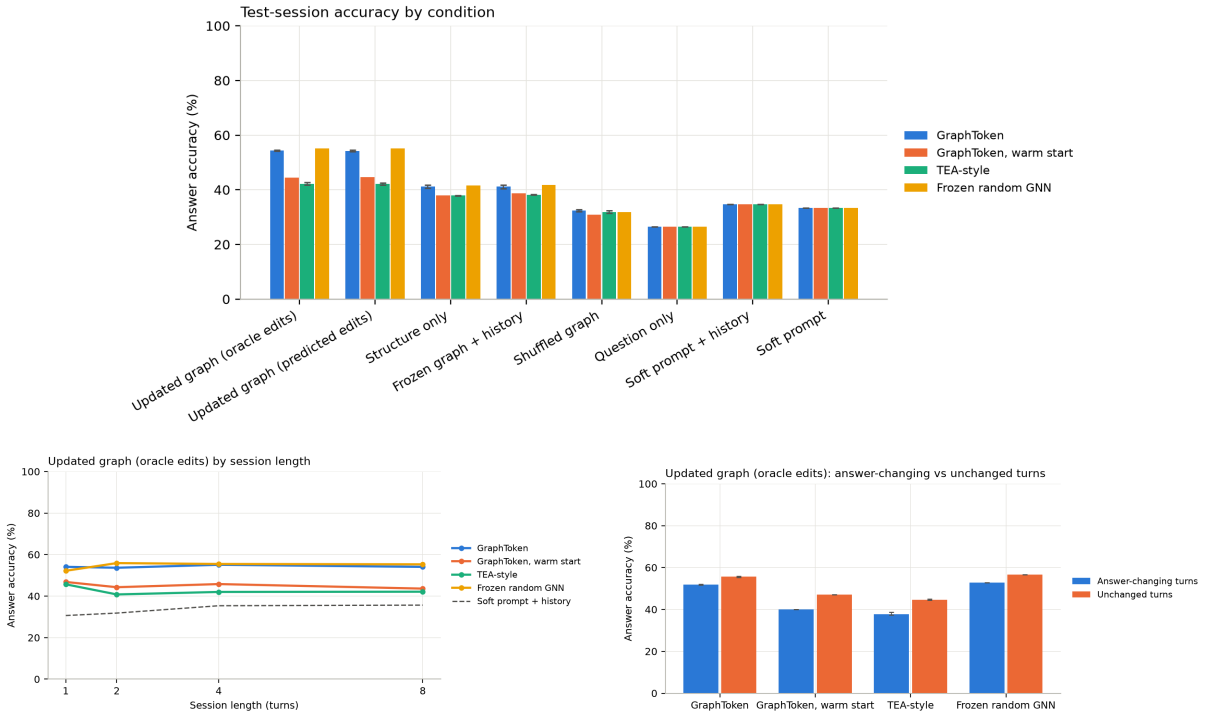


Figure 3: Test results for all four graph readers. Top: accuracy under each condition. Bottom left: `oracle_updated_graph` by session length, with the soft prompt. Bottom right: `oracle_updated_graph` on answer-changing and unchanged turns. GraphToken-style and TEA-style are means of three seeds (error bars: seed range).

Task	GT	TEA	Warm	Rand.	Soft	Maj.	Local
node_degree	99.1	44.2	75.0	94.9	11.6	11.4	100.0
cycle_membership	79.0	52.0	48.0	81.8	51.8	50.5	95.7
reachability	65.7	61.2	62.6	67.4	54.0	54.0	92.4
edge_exists	61.2	56.4	59.6	65.9	45.7	55.3	95.2
constrained_reachability	56.6	51.7	55.8	57.1	50.0	50.0	65.4
shortest_path	33.6	23.0	26.5	34.3	14.1	14.4	62.1
filtered_neighbor_count	32.4	27.6	30.8	31.3	18.2	15.7	29.0
within_hops_count	18.0	17.8	18.2	19.2	10.9	12.1	20.7
most_common_attr. within hops	28.5	36.3	37.9	30.1	26.5	6.3	–
attribute_lookup	29.5	46.5	46.2	28.0	25.8	5.8	–
attribute_check	49.4	52.9	53.8	48.7	50.5	50.5	–

Table 7: Static test accuracy (%) by task (396 items per task). GT and TEA are means of three seeds. Local: learned local ceiling (better of the one- and two-hop classifiers; structural tasks only).

Projector learning-rate selection. For each graph reader, seed 42 was trained with projector learning rates 10^{-3}, 3×10^{-4}, and 10^{-4}, and the rate with the lowest best validation loss was kept (Table 9); test data was never used. All four chose 10^{-4}. At 10^{-3}, the readers whose projector reads a frozen or pretrained GNN spiked at the peak learning rate and had not converged after one epoch.

G Benchmark Construction

Topology. Each graph is built for a chosen station count (12–22) and density. A core of at least five stations forms a ring. The remaining stations go into three side chains (four for graphs with 17 or more stations) of length one to four; dense graphs and graphs with 20 or more stations use chains of at most two, medium graphs at most three. A chain of length two or more ends in a triangle with probability 0.6, which creates small cycles off the ring. About two thirds of the ring stations form a hub; random chords are added only between hub stations until the edge count reaches the target mean degree (2.8, 3.8, or 5.0), and each chain attaches to a random hub station. This gives three kinds of degree-2 station: ring stations out-

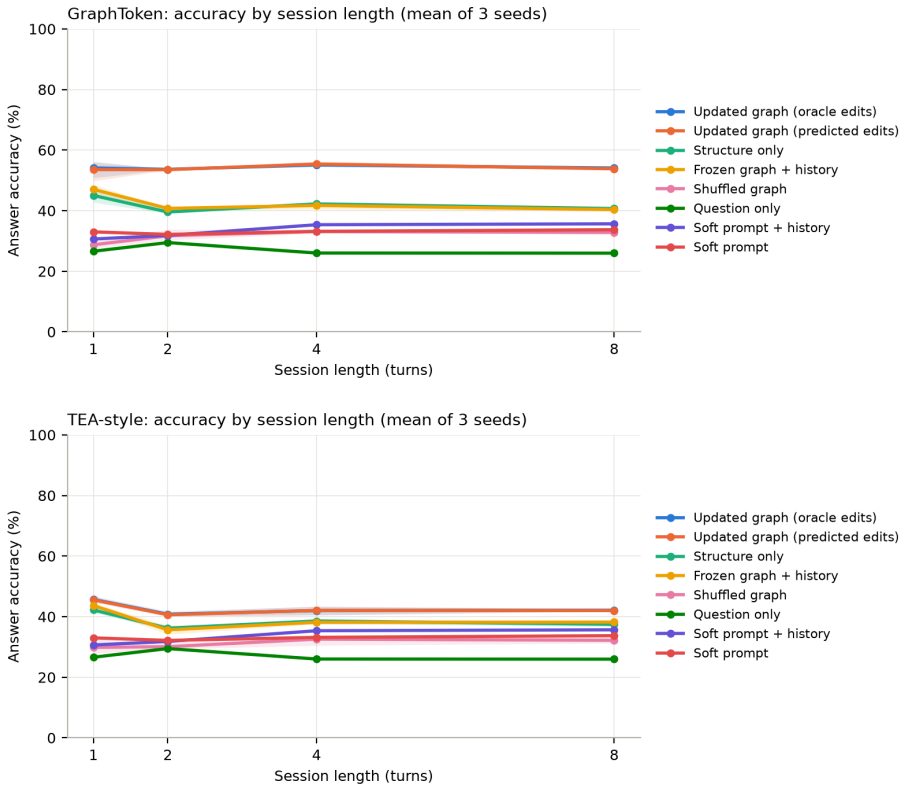


Figure 4: Test accuracy by session length for all conditions, GraphToken-style (top) and TEA-style (bottom).

Item	Value
LLM	Llama-3.1-8B-Instruct, bf16, frozen
Node features	`bert-base-uncased`, 768-d, frozen, mean-pooled over the node text
GNN	GraphSAGE 768/1024/1024/4096, sum aggregation, LayerNorm, ReLU, dropout 0
Readout	pooled / source / target embeddings, hop scalar, task one-hot (12,305-d)
Projector	$12{,}305 \to 10 \times 4096$, linear
Prompt	`Question: {q}\nAnswer:` followed by the graph tokens
TEA-GLM pretraining	Adam $2{\times}10^{-3}$, τ=0.4, dropout 0.5, edge drop 0.3/0.4, feature mask 0/0.1, 1,000 principal components, 32 graphs/batch, $\leq$70 epochs, patience 10
Training	AdamW, wd 10^{-4}, eff. batch 24 (8×3), bf16, 1 epoch, cosine, 200 warmup, min lr ratio 0.05
Training data	18,000 static items + 6,000 before/after pairs around a random edit
Projector lr	10^{-4} for all four graph readers
GNN lr (joint)	10^{-4} (GraphToken-style, warm start)
Soft prompt	10×4096, lr 10^{-3}, final checkpoint
Checkpoint	lowest validation loss, every 100 steps on 1,000 validation items
Evaluation	greedy, at most 32 new tokens, first non-empty line scored

Table 8: Key hyperparameters.

Reader	10^{-3}	$3{\times}10^{-4}$	10^{-4}
GraphToken-style	0.486	0.481	0.476
TEA-style	0.874	0.605	0.572
Warm start	0.761	0.534	0.523
Frozen random GNN	0.781	0.505	0.485

Table 9: Best validation loss (seed 42) by projector learning rate; lower is better.

side the hub (on a cycle), chain stations (not on a cycle), and stations in an end triangle (on a small cycle). A graph is kept only if it is connected and its diameter is within the cap (7, 6, or 5); otherwise it is regenerated, up to 2,000 times.

Attributes. Lines are grown by breadth-first search from random seed stations (three lines for graphs of up to 14 stations, four otherwise), so each line is a connected region; an edge within a line is a `track`, an edge between lines a `transfer`. Stations get fictional two-word names and independently drawn attributes: disabled access and

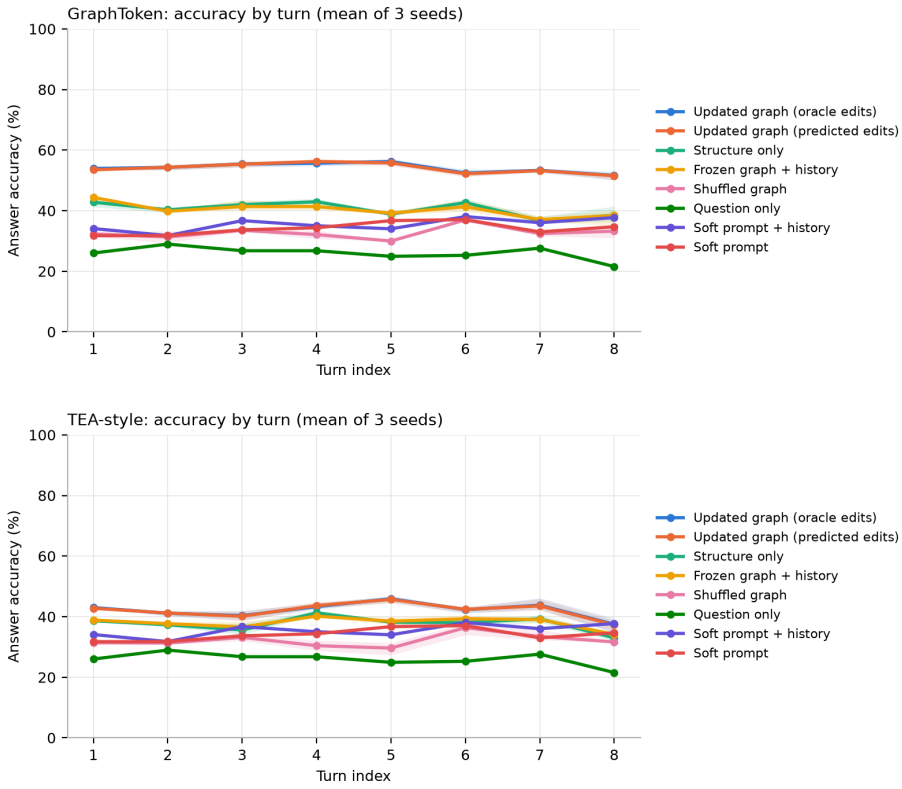


Figure 5: Test accuracy by turn index for all conditions, GraphToken-style (top) and TEA-style (bottom). Later turns occur only in 8-turn sessions.

rail (coin flips), architecture, cleanliness, music, and size. One to three stations are closed. Edges carry line colour, stroke, air conditioning, and build year, derived from the endpoint IDs so that replaying an edit gives the same edge. Node IDs are a random permutation of the generation order, and the node and edge lists are shuffled separately, so neither IDs nor list order reveal the structure.

Sessions. Each turn applies an edit program of up to three operations and asks one question. About 17% of turns are NOOP; no session contains a net-no-op, a cancelled edit, or an edit that disconnects the graph, deleted edges are never bridges, and at most three stations are closed at once. A per-split quota prefers answer-changing questions (target share 0.45), which gives 36.4% answer-changing test turns, and questions are balanced on whether they name an edited station (50.2% do). Revisions come from three wording families (`direct`, `service`, `notice`); `notice` is held out of the few-shot edit prompt. Each task uses exactly one family (495 turns each), so any accuracy split by family is a task-mix split, not a wording effect.

Gold operations	**Turns**
NOOP	745
ADD EDGE	608
ADD EDGE + DEL EDGE	591
ADD EDGE + SET NODE	577
DEL EDGE	571
DEL EDGE + SET NODE	540
SET NODE	539
ADD EDGE + DEL EDGE + SET NODE	284

Table 10: Edit-operation mix of the 4,455 test turns.

Fixes in generator v4.1. An earlier version had only 14.6% answer-changing turns (the stale oracle scored about 85%); the quota above fixes this. Without the end triangles, the rule "a neighbour of a leaf is not on a cycle" solved about 95% of cycle questions; with them it scores 79%. Cycle questions always use the global hop tag, and the before-edit training items pass the same filters as the test items. Every stored label and edit was re-derived by independent solvers (0 mismatches), and a leak audit of the question text is clean.

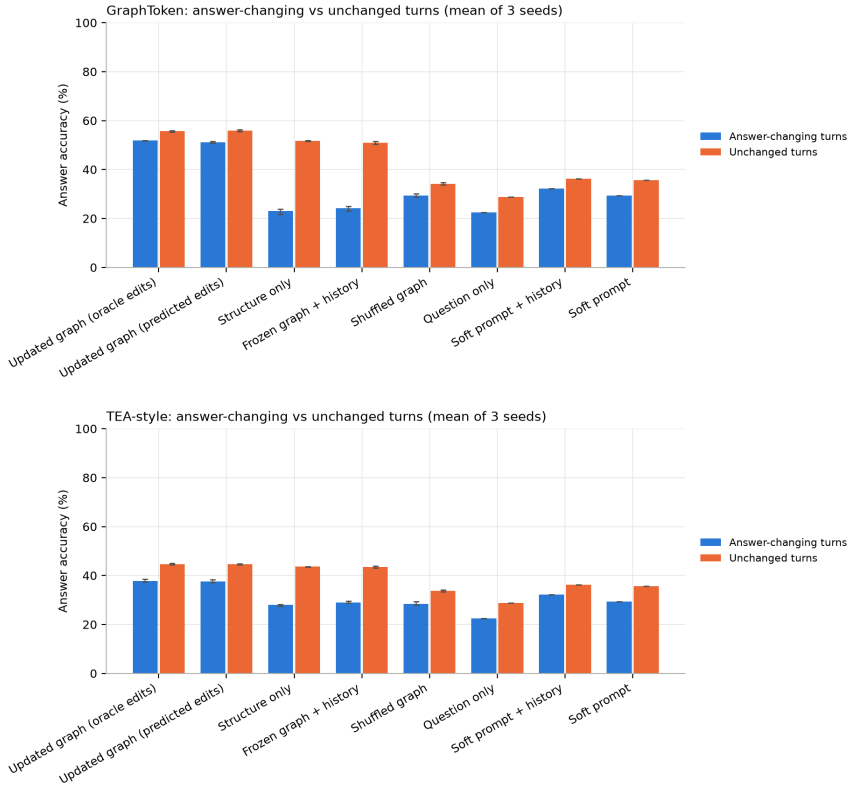


Figure 6: Test accuracy on answer-changing and unchanged turns for all conditions, GraphToken-style (top) and TEA-style (bottom).

H Local-Shortcut Ceilings and Text Lookups

One gradient-boosted classifier per task is fit on static train. The one-hop version sees the degree and status of the queried stations, their numbers of leaf and closed neighbours, the smallest neighbour degree, the hop radius, the graph size, and the number of closed stations. The two-hop version adds stations, leaves, and closed stations at two hops, triangles, the largest and summed neighbour degree, and common neighbours; it sees no distances. Over the non-attribute tasks, the one-hop classifier reaches 63.4% on static test and 61.9% on the updated test-session graphs, and the two-hop classifier 70.1% and 67.0%. Table 11 gives the better of the two per task.

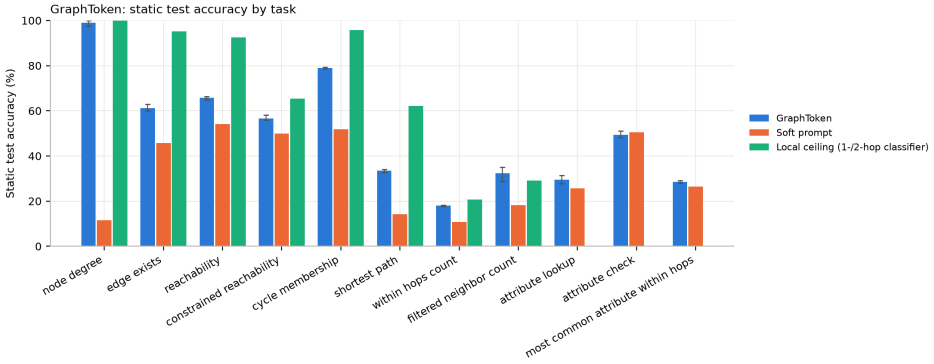
Task	Static test	Updated graphs
node_degree	100.0	100.0
cycle_membership	95.7	88.1
edge_exists	95.2	91.5
reachability	92.4	91.3
constrained_reachability	65.4	64.4
shortest_path	62.1	55.6
filtered_neighbor_count	29.0	33.7
within_hops_count	20.7	16.8

Table 11: Learned local ceiling (%) per task: the better of the one- and two-hop classifiers.

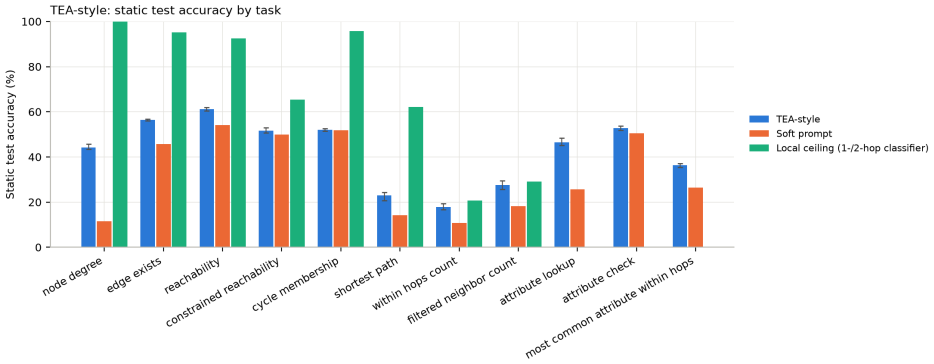
Two text lookups are fit on validation sessions without any graph. `text_only` uses only what the revision text says about the queried stations and scores 36.6% overall, 41.4% on answer-changing turns, and 28.4% on the changed-and-unmentioned turns. `stale_plus_text` adds the pre-edit answer and scores 78.7%, 54.3%, and 2.5%. Both score high overall because half of the turns ask about a station that was just edited; the changed-and-unmentioned slice removes that route.

I Additional Multi-turn Results

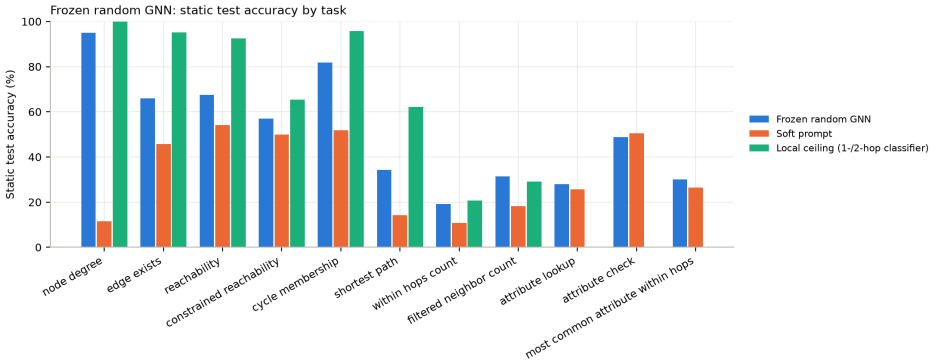
Per-run intervals and paired differences. Table 12 gives each run with its 95% session-clustered interval. Within every GraphToken-style seed, re-encoding beats stale tokens plus history by 12.8–13.6 points, stale tokens alone by 12.8–13.4, and the shuffled graph by 21.7–22.0; within every TEA-style seed the gains are 3.7–4.5 and 9.4–10.9. Oracle minus predicted lies between -0.1 and $+0.2$ in every run, and every interval includes zero. Across runs (paired



(a) GraphToken-style



(b) TEA-style



(c) Frozen random GNN

Figure 7: Static test accuracy on all eleven task families, with the soft prompt and the learned local ceiling. GraphToken-style and TEA-style are means of three seeds (error bars: seed range).

by turn), GraphToken-style beats the soft prompt by 19.4–19.8 points with the revision history and 20.8–21.2 without it, and TEA-style by 11.5–12.7. GraphToken-style minus the frozen random GNN is −1.1 (CI −2.3 to 0.0), so the two are tied; the warm start is 9.6 points below GraphToken-style (CI 7.7–11.5), and TEA-style is 13.2 below the frozen random GNN (CI 11.0–15.4).

Session length and density. Tables 13 and 14 give the numbers behind Section 6.3 and Section 6.5. GraphToken-style re-encoded accuracy stays within 53.6–55.1% at every length and

Run	Oracle	Predicted	Stale + hist.	Shuffled
GT s42	54.1 [51.8, 56.3]	54.0	40.5	32.1
GT s43	54.5 [52.2, 56.8]	54.5	41.7	32.8
GT s44	54.3 [51.9, 56.4]	54.0	41.4	32.5
TEA s42	42.0 [40.2, 43.9]	41.8	38.3	31.4
TEA s43	41.8 [40.0, 43.7]	41.9	38.1	32.4
TEA s44	42.6 [40.8, 44.6]	42.5	38.1	31.7
Warm s42	44.5 [42.6, 46.5]	44.6	38.8	30.8
Random s42	55.2 [52.9, 57.5]	55.1	41.7	31.8

Table 12: Test accuracy (%) per run, with the 95% session-clustered bootstrap interval of the oracle condition (intervals of the other columns are of similar width, about ±2 points).

Condition	1	2	4	8
GT oracle	54.1	53.6	55.1	54.1
GT predicted	53.5	53.5	55.4	53.8
GT stale + history	47.0	40.7	41.7	40.3
GT structure only	45.0	39.6	42.2	40.7
GT shuffled	28.7	31.7	33.1	32.8
TEA oracle	45.7	40.8	42.0	42.1
TEA stale + history	43.7	35.6	38.1	38.2
Warm oracle	46.8	44.3	45.8	43.6
Random oracle	52.2	55.9	55.5	55.3
Random stale + history	45.1	39.6	42.3	41.5
soft_prompt	30.6	31.8	35.4	35.6
serialized_current_graph	31.6	34.7	29.6	30.7
serialized_initial_history	29.3	33.3	29.0	28.7
question_only	26.6	29.5	26.0	26.0
stale oracle (reference)	64.0	61.1	63.3	64.3

Table 13: Test accuracy (%) by session length. GT and TEA are means of three seeds. The stale oracle runs no model; its score is the share of unchanged turns at each length.

Condition	Sparse	Medium	Dense
GT oracle	53.0	54.5	55.4
GT predicted	52.8	54.4	55.4
GT stale + history	39.6	41.7	42.3
GT shuffled	32.3	33.6	31.4
TEA oracle	42.7	42.6	41.1
Warm oracle	45.3	44.4	43.8
Random oracle	53.4	56.5	55.8
soft_prompt	34.3	33.9	36.0
serialized_current_graph	33.7	31.3	28.0
question_only	29.2	26.3	23.9

Table 14: Test accuracy (%) by graph density.

within 53.0–55.4% at every density. Stale tokens plus history drop from 47.0% in one-turn sessions to 40.3–41.7% in longer ones; the share of answer-changing turns is 36–39% at every length.

By task. Table 15 breaks the multi-turn results down by task. Re-encoding helps most on node degree (99.9% against 40.8% for stale tokens plus history), cycle membership (83.4% against 62.5%), and edge existence (66.3% against 53.2%). GraphToken-style is far below the local ceiling on shortest path (25.5% against 55.6%) and reachability (68.4% against 91.3%), reaches it on the two counting tasks, and on the most-common-attribute task equals the shuffled graph (28.6%). The frozen random GNN matches or beats GraphToken-style on every structural task, while TEA-style is at chance on cycle membership and is the strongest reader on the most-common-attribute task.

Local-wrong slice. On the 1,052 test turns that both local classifiers get wrong, the question alone scores 24.6%. Every graph reader is below it with re-encoded tokens: GraphToken-style 22.2% (s.d. 0.3; shuffled graph 21.2%), TEA-style 23.6%, the warm start 21.6%, and the frozen random GNN 19.2%; the soft prompt scores 18.6%. The two serialized-graph baselines score slightly above the question alone on this slice (26.8% and 27.9%), so reading the whole graph as text helps a little on these turns, although these baselines are weak overall.

J Write Path

The edit model is the same frozen LLM for every reader, so the predicted edits are identical in every run. It writes an execution-equivalent program on 96.6% of test turns, and the graph built from its own edits is exactly right on 91.1% of turns (Table 16). NOOP turns are always correct and status and deletion edits are above 97%; the weakest case is a lone `ADD EDGE`. The held-out `notice` family is harder, but each task uses a single family, so this gap mixes wording with the edits the `notice` tasks draw. For GraphToken-style, the predicted condition scores 54.2% against 54.3% for the oracle (paired CI of seed 42: -0.2 to 0.4), and the two agree within 1.2 points by session length, change status, density, and task for every reader.

K GNN Probes

Method. For each encoder we compute the node embeddings of 600 train-split and 396 test-split static graphs once. A multinomial logistic regression (full-batch L-BFGS on standardized features, small weight decay) is fit on the train graphs and scored on the test graphs. The targets are node degree (capped at 6), station status, and the number of other stations within two hops (capped at 8). Two collapse measures describe the embeddings: the mean cosine similarity between nodes of the same graph, and the effective rank, the participation ratio of the covariance spectrum (roughly, the number of directions that carry most of the variance).

Readings. Table 17 gives the results.

- **BERT input.** Status is fully decodable (100%) because it is in the node text, but de-

Task	GT	TEA	Warm	Rand.	Stale+h.	Shuf.	Soft	Local
node_degree	99.9	49.6	72.7	97.6	40.8	11.1	11.7	100.0
cycle_membership	83.4	49.2	43.4	87.3	62.5	54.5	58.0	88.1
reachability	68.4	60.2	63.2	70.3	61.0	53.7	59.2	91.3
edge_exists	66.3	58.1	58.0	70.5	53.2	46.9	55.4	91.5
constrained_reachability	61.0	56.6	55.4	61.8	52.9	47.3	50.7	64.4
filtered_neighbor_count	34.3	30.6	31.5	33.1	30.0	20.1	18.2	33.7
most_common_attr. within hops	28.6	36.0	34.7	33.1	29.6	28.6	27.7	–
shortest_path	25.5	20.5	22.2	24.8	24.0	15.1	15.2	55.6
within_hops_count	21.3	18.6	19.2	18.4	17.0	14.7	16.6	16.8

Table 15: Multi-turn test accuracy (%) by task (n=495 each). GT, TEA, Warm, Rand.: re-encoded gold graph; Stale+h. and Shuf.: GraphToken-style stale tokens plus history and shuffled graph; Soft: soft prompt with history; Local: learned local ceiling on the updated graphs. GT and TEA are means of three seeds.

Gold operations	Edit acc.	Turns
NOOP	100.0	745
ADD EDGE	79.9	608
ADD EDGE + DEL EDGE	99.2	591
ADD EDGE + SET NODE	99.0	577
DEL EDGE	99.5	571
DEL EDGE + SET NODE	99.6	540
SET NODE	97.6	539
ADD + DEL EDGE + SET NODE	99.6	284
Family `direct`	99.3	1,485
Family `service`	98.9	1,485
Family `notice` (held out)	91.5	1,485

Table 16: Execution-equivalent edit accuracy (%) of the frozen LLM on test turns, by gold operations and by wording family.

Node embeddings	Degree	Status	2-hop	Cos.	Rank
BERT input (no GNN)	32.2	100.0	57.9	0.97	13.0
Frozen random GNN	100.0	96.6	92.3	0.92	1.9
GraphToken-style GNN	100.0	95.8	96.9	0.97	1.9
TEA-GLM-pretrained GNN	96.4	88.1	79.2	0.24	16.2
Warm-start GNN	98.3	88.1	82.1	0.37	12.9
Majority class	36.5	88.2	59.0		

Table 17: Linear probes on frozen node embeddings (test-split graphs, accuracy %). Cos.: mean cosine similarity between nodes of a graph; Rank: effective rank of the embedding covariance. GraphToken-style and warm start are seed 42.

gree is not (32.2%, below the 36.5% majority class).

- **Frozen random GNN.** Sum aggregation makes degree (100%), status (96.6%), and the two-hop count (92.3%) linearly decodable without any training.
- **GraphToken-style GNN.** Joint training changes little (100%, 95.8%, 96.9%); only the two-hop count improves.
- **TEA-GLM-pretrained GNN.** Status falls to 88.1%, which is the majority class (88.2%), so status is lost. Degree falls slightly (96.4%) and the two-hop count more (79.2%).
- **Warm start.** One epoch of joint training from the pretrained GNN brings degree back (98.3%) but not status (88.1%), and the two-hop count only partly (82.1%).
- **Collapse.** The TEA-GLM embeddings are the least collapsed (cosine 0.24, rank 16.2), while the random and GraphToken-style embeddings are nearly collapsed (rank 1.9) and still decode every target. Collapse statistics therefore do not measure what a linear projector can read.

These probe results match the multi-turn task profile. The frozen random GNN and GraphToken-style answer 97.6% and 99.9% of degree questions, TEA-style 49.6%, and the warm start 72.7%. On cycle membership the first two reach 87.3% and 83.4%, while TEA-style (49.2%) and the warm start (43.4%) are at or below chance.

L Zero-shot Text Baselines

The serialized-graph prompts list every station with its attributes and every edge with its relation, followed by the question; at most 1,937 Llama tokens, they fit the 4,096-token budget untruncated. Zero-shot Llama reading the current graph scores 31.0%, close to the per-task majority (31.8%), and reading the initial graph plus every revision scores 29.4%. These baselines use the plain `Question: ... Answer:` template, see no examples of the answer formats the readers are trained on (path length counting both endpoints, `unreachable`, bare integers), and must map station names to the node IDs used in the edge list. They are therefore a floor for reading the graph as text, not its best case.